Building Agentic AI with Ollama: A Practical Guide

Welcome, future AI systems engineer! This guide will equip you with the practical knowledge to leverage Ollama as the inference backbone for building robust, production-grade agentic applications locally. We'll move beyond theoretical concepts and dive into hands-on implementation, focusing on how Ollama empowers your agents to interact with the real world.

1. Foundation: Ollama's Role in an Agentic System

Ollama is more than just a model runner; it's your local **inference backbone** for agentic AI. It allows you to run large language models (LLMs) directly on your machine, providing a powerful, private, and cost-effective engine for your agents.

Why Local Models Matter for Agents

Running LLMs locally with Ollama offers significant advantages, especially when building agentic systems that require frequent, low-latency interactions and data privacy:

- **Privacy & Security:** Your data never leaves your machine. This is paramount for agents handling sensitive information or operating in regulated environments.
- **Latency:** Local inference eliminates network overhead, leading to faster response times. For agents performing rapid decision-making or real-time tasks, this is critical.
- **Cost-Effectiveness:** Once downloaded, local models incur no per-token API costs, making extensive experimentation and high-volume inference economically viable.
- **Control & Customization:** You have full control over the model, its environment, and can easily switch between different models or even fine-tune them (though fine-tuning itself is outside this guide's scope).

Ollama vs. API-Based LLMs (e.g., OpenAI, Claude)

While API-based LLMs offer convenience and access to cutting-edge models, Ollama provides a distinct set of trade-offs that are often beneficial for agent development:

Feature	Ollama (Local Models)	API-Based LLMs (e.g., OpenAI, Claude)
Deployment	On-premise, local machine	Cloud-hosted, accessed via API

Data Privacy	High (data stays local)	Depends on provider's policy, data often processed in cloud
Latency	Low (no network overhead)	Moderate to High (network-dependent)
Cost	Zero per-token cost (after hardware/electricity)	Per-token billing, can be expensive for high usage
Model Choice	Growing open-source ecosystem	Proprietary, often state-of-the-art
Control	Full control over model and environment	Limited control, dependent on API provider
Setup	Requires local hardware setup	Minimal setup, just API key

Pain Point Solved: Ollama addresses the pain points of **data privacy, operational cost, and latency** that often arise when developing and deploying agentic systems reliant on external LLM APIs.

Model Selection Strategy

Choosing the right model is a critical decision that impacts your agent's performance, speed, and resource consumption. It's a balance between capability, size, and speed.

- **Capability:** Larger models generally offer better reasoning and language understanding. For complex agent tasks, you might need a more capable model.
- **Size:** Model size directly correlates with VRAM/RAM requirements and download time. Smaller models are faster and run on less powerful hardware.
- **Speed:** Inference speed depends on model size, your hardware (CPU vs. GPU), and quantization (e.g., `q4_0`, `q8_0`).

Recommendation: Start with a moderately sized, capable model like `llama2:7b` or `mistral:7b`. These offer a good balance for many agentic tasks on consumer hardware. Experiment with different quantizations (e.g., `llama2:7b-chat-q4_0`) to find the sweet spot for your system.

Pain Point Solved: Navigating the trade-offs in model selection to ensure your agent has the right balance of intelligence and efficiency for its operating environment.

2. Core Capabilities: Getting Started with Ollama

Running Models Locally

Ollama simplifies running LLMs locally. Once installed, you interact with it via the `ollama` CLI or its local API endpoint.

`ollama run` - Interactive Mode

This command pulls a model if not already present and starts an interactive chat session.

Practical Implementation:

```bash
# Pull and run the Mistral 7B model
ollama run mistral

# You can then chat with it:
>>> hi
Hello! How can I help you today?
>>> What is the capital of France?
Paris is the capital of France.
```

Why it matters for agents: Great for quick testing of model behavior and understanding its conversational style before integrating into an agent.

Common Pitfalls:

- **Model not found:** Ensure you type the model name correctly (e.g., `mistral` not `Mistral`).
- **Resource exhaustion:** Running very large models on insufficient RAM/VRAM can lead to slow performance or crashes. Check `ollama ps` to see running models and `ollama rm <model_name>` to remove unused ones.

`ollama serve` - API Mode

For agentic applications, you'll primarily interact with Ollama via its REST API. `ollama serve` starts the Ollama server, typically on `http://localhost:11434`.

Practical Implementation:

```bash
# Start the Ollama server (usually runs in the background )
ollama serve
```

Why it matters for agents: This is the gateway for your Python backend agents to programmatically send prompts and receive responses from local LLMs.

Common Pitfalls:

- **Port conflicts:** If another service is using port 11434, Ollama might fail to start. You can configure Ollama to use a different port via environment variables (e.g., `OLLAMA_HOST=0.0.0.0:8000 ollama serve`).
- **Not running:** Your agent won't be able to connect if `ollama serve` isn't running. Always ensure it's active before starting your agent.

`ollama pull` - Pre-downloading Models

It's good practice to pull models beforehand, especially for deployment or when internet access might be unreliable.

Practical Implementation:

```bash
# Pull a specific model
ollama pull llama2:7b

# Pull a specific quantization of a model
ollama pull mistral:7b-instruct-v0.2-q4_K_M
```

Why it matters for agents: Ensures your agent can start quickly without waiting for model downloads, crucial for production environments.

Common Pitfalls:

- **Disk space:** Models can be several gigabytes. Ensure you have enough disk space before pulling many models.
- **Network issues:** Large downloads can be interrupted. Ollama usually resumes, but a stable connection is best.

API Integration (REST Endpoint, Python Client)

Your Python agents will communicate with the running Ollama server via its REST API. The `ollama` Python library simplifies this interaction.

Pain Point Solved: Providing a programmatic interface for agents to send prompts and receive LLM responses, abstracting away the HTTP requests.

Practical Implementation (Python):

First, install the Ollama Python client:

```bash
```

```
sudo pip3 install ollama
```

Then, in your Python code (e.g., `agent_core.py`):

Python

```python
import ollama

def get_llm_response(prompt: str, model: str = "mistral") -> str:
    try:
        response = ollama.chat(
            model=model,
            messages=[
                {
                    "role": "user",
                    "content": prompt,
                }
            ],
        )
        return response["message"]["content"]
    except Exception as e:
        print(f"Error getting LLM response: {e}")
        return ""

if __name__ == "__main__":
    # Ensure 'ollama serve' is running in your terminal
    # and 'mistral' model is pulled (ollama pull mistral)
    user_query = "What is the capital of Canada?"
    answer = get_llm_response(user_query)
    print(f"Agent: {answer}")

    user_query_2 = "Explain the concept of recursion in programming."
    answer_2 = get_llm_response(user_query_2)
    print(f"Agent: {answer_2}")
```

Test it:

1. Ensure `ollama serve` is running in a separate terminal.
2. Ensure `mistral` model is pulled (`ollama pull mistral`).
3. Save the Python code above as `agent_core.py`.
4. Run `python3.11 agent_core.py`.

Expected Output:

Plain Text

```
Agent: The capital of Canada is Ottawa.
Agent: Recursion in programming is a technique where a function calls itself...
```

Common Pitfalls:

- **Ollama server not running:** The `ollama.chat` call will fail with a connection error. Always verify `ollama serve` is active.
- **Model not pulled:** If the specified `model` is not available locally, the API call will fail. Use `ollama pull <model_name>`.
- **Incorrect API usage:** Refer to the `ollama` Python library documentation for correct function signatures and parameters, especially for chat vs. completion endpoints.

Model Performance Tuning

Optimizing model performance involves adjusting parameters to control output quality, creativity, and adherence to constraints.

Pain Point Solved: Tailoring the LLM's behavior to specific agent tasks, balancing creativity with factual accuracy and ensuring outputs fit desired formats.

Context Window

Essence: The context window (or context length) defines how much text (tokens) the model can process at once, including your prompt and its generated response. It's the model's **short-term memory**.

Why it matters for agents: Agents often need to process long documents or maintain extensive conversation history. Understanding and managing the context window is crucial to avoid truncation and ensure the agent has all necessary information.

Practical Implementation:

When pulling a model, you can often see its default context window. When making API calls, you manage the input length.

Python

```
import ollama

def get_llm_response_with_context(prompt: str, model: str = "mistral", max_toke
    try:
        response = ollama.chat(
            model=model,
            messages=[
                {
                    "role": "user",
```

```
                    "content": prompt,
                }
            ],
            options={
                "num_predict": max_tokens, # Max tokens to generate
            }
        )
        return response["message"]["content"]
    except Exception as e:
        print(f"Error getting LLM response: {e}")
        return ""

if __name__ == "__main__":
    long_prompt = """
    Summarize the following article in detail, ensuring all key points are cove
    The article discusses the history of artificial intelligence from its philo
    roots to modern deep learning breakthroughs. It covers early symbolic AI, e
    the AI winter, connectionism, neural networks, and the rise of large langua
    Pay particular attention to the ethical implications discussed at each stag
    """
    # If the article content itself was very long, we'd need to ensure it fits
    # For this example, we're just limiting the output length.
    summary = get_llm_response_with_context(long_prompt, max_tokens=200)
    print(f"Summary (first 200 tokens): {summary}")
```

Common Pitfalls:

- **Truncation:** Sending a prompt that exceeds the model's context window will result in the model ignoring the overflow. Always monitor input length, especially with conversation history or long documents.
- **Misunderstanding `num_predict`:** This parameter limits the *output* tokens, not the *input* context. You must manage input length separately.

Temperature

Essence: Temperature controls the randomness of the model's output. A higher temperature (e.g., 0.8-1.0) leads to more creative and diverse responses, while a lower temperature (e.g., 0.1-0.3) makes the output more deterministic and focused.

Why it matters for agents: For tasks requiring creativity (e.g., brainstorming, content generation), a higher temperature is suitable. For tasks needing factual accuracy and consistency (e.g., data extraction, code generation), a lower temperature is preferred.

Practical Implementation:

Python

```
import ollama

def get_llm_response_with_temp(prompt: str, model: str = "mistral", temperature
    try:
        response = ollama.chat(
            model=model,
            messages=[
                {
                    "role": "user",
                    "content": prompt,
                }
            ],
            options={
                "temperature": temperature,
            }
        )
        return response["message"]["content"]
    except Exception as e:
        print(f"Error getting LLM response: {e}")
        return ""

if __name__ == "__main__":
    creative_prompt = "Write a short, imaginative story about a robot who disco
    creative_story = get_llm_response_with_temp(creative_prompt, temperature=0.
    print(f"Creative Story (Temp=0.9):\n{creative_story}\n")

    factual_prompt = "List three key features of Python's `asyncio` library."
    factual_response = get_llm_response_with_temp(factual_prompt, temperature=0
    print(f"Factual Response (Temp=0.1):\n{factual_response}")
```

Common Pitfalls:

- **Too high temperature for factual tasks:** Can lead to hallucinations or nonsensical output.
- **Too low temperature for creative tasks:** Can result in repetitive or uninspired responses.

Sampling (Top-K, Top-P)

Essence: These parameters control how the model selects the next word (token) from its probability distribution. `top_k` limits the selection to the top K most probable tokens, while `top_p` (nucleus sampling) selects from the smallest set of tokens whose cumulative probability exceeds P.

Why it matters for agents: Fine-tuning these can further control the diversity and quality of the output, especially in conjunction with temperature. For agents, you often want a

balance between coherence and avoiding repetition.

Practical Implementation:

```python
import ollama

def get_llm_response_with_sampling(prompt: str, model: str = "mistral", top_k: 
    try:
        response = ollama.chat(
            model=model,
            messages=[
                {
                    "role": "user",
                    "content": prompt,
                }
            ],
            options={
                "top_k": top_k,
                "top_p": top_p,
            }
        )
        return response["message"]["content"]
    except Exception as e:
        print(f"Error getting LLM response: {e}")
        return ""

if __name__ == "__main__":
    sampling_prompt = "Describe a futuristic city with flying cars and vertical
    sampling_response = get_llm_response_with_sampling(sampling_prompt, top_k=5
    print(f"Sampling Response:\n{sampling_response}")
```

Common Pitfalls:

- **Over-constraining:** Setting `top_k` or `top_p` too low can make the model repetitive or get stuck in loops.
- **Under-constraining:** Setting them too high (or not at all) can lead to less coherent or off-topic responses.

Prompt Engineering for Agents

Prompt engineering is the art and science of crafting effective inputs to guide the LLM's behavior. For agents, this is crucial for reliable decision-making and structured output.

Pain Point Solved: Ensuring the LLM consistently produces desired outputs, follows instructions, and integrates seamlessly into an agent's workflow.

System Prompts

Essence: A system prompt defines the role, persona, and general instructions for the LLM. It sets the overall context and constraints for the entire interaction.

Why it matters for agents: It establishes the agent's identity and core directives, ensuring it behaves consistently across different tasks. For example, an agent might always act as a helpful assistant or a strict code reviewer.

Practical Implementation:

Python

```
import ollama

def get_agent_response(user_message: str, system_message: str, model: str = "mi
    try:
        response = ollama.chat(
            model=model,
            messages=[
                {"role": "system", "content": system_message},
                {"role": "user", "content": user_message},
            ],
        )
        return response["message"]["content"]
    except Exception as e:
        print(f"Error getting agent response: {e}")
        return ""

if __name__ == "__main__":
    code_reviewer_system_prompt = "You are a senior Python code reviewer. Your
    code_to_review = "def add(a, b): return a + b"
    review = get_agent_response(code_to_review, code_reviewer_system_prompt)
    print(f"Code Review:\n{review}")

    creative_writer_system_prompt = "You are a whimsical storyteller. Your resp
    story_prompt = "Tell me a short story about a lost sock."
    story = get_agent_response(story_prompt, creative_writer_system_prompt)
    print(f"\nWhimsical Story:\n{story}")
```

Test it:

1. Ensure `ollama serve` is running and `mistral` is pulled.
2. Save the Python code as `agent_prompts.py`.
3. Run `python3.11 agent_prompts.py`.

Expected Output: (Will vary slightly due to model randomness)

```
Code Review:
Consider adding type hints for 'a' and 'b' to improve readability and maintainal

Whimsical Story:
Once upon a time, in a laundry basket filled with forgotten dreams, lived a lon
```

Common Pitfalls:

- **Vague system prompts:** If the system prompt is not clear, the model might not adhere to the desired persona or instructions. Be specific.
- **Conflicting instructions:** Avoid giving contradictory instructions in the system and user prompts, as this can confuse the model.

Few-Shot Examples

Essence: Few-shot examples involve providing a few input-output pairs within the prompt to demonstrate the desired behavior or format. It teaches the model by example.

Why it matters for agents: When you need the LLM to follow a very specific format or perform a particular type of transformation, few-shot examples are more effective than just verbal instructions.

Practical Implementation:

```python
import ollama

def get_formatted_output(text_input: str, model: str = "mistral") -> str:
    prompt = f"""
    Extract the name and email from the following text and return it in JSON fo

    Example 1:
    Text: "Contact John Doe at john.doe@example.com for further details."
    Output: {{"name": "John Doe", "email": "john.doe@example.com"}}

    Example 2:
    Text: "Please reach out to Jane Smith (jane.smith@company.org) for support.
    Output: {{"name": "Jane Smith", "email": "jane.smith@company.org"}}

    Text: "{text_input}"
    Output:
    """
    try:
        response = ollama.chat(
```

```
            model=model,
            messages=[
                {"role": "user", "content": prompt},
            ],
            options={
                "temperature": 0.01, # Keep temperature low for structured outp
            }
        )
        return response["message"]["content"]
    except Exception as e:
        print(f"Error getting formatted output: {e}")
        return ""

if __name__ == "__main__":
    text_data_1 = "The project lead is Alice Wonderland, her email is alice@exa
    extracted_data_1 = get_formatted_output(text_data_1)
    print(f"Extracted Data 1:\n{extracted_data_1}\n")

    text_data_2 = "For inquiries, email bob.builder@construction.com, attention
    extracted_data_2 = get_formatted_output(text_data_2)
    print(f"Extracted Data 2:\n{extracted_data_2}")
```

Test it:

1. Ensure `ollama serve` is running and `mistral` is pulled.
2. Save the Python code as `few_shot_agent.py`.
3. Run `python3.11 few_shot_agent.py`.

Expected Output: (Should be consistent JSON)

```JSON
Extracted Data 1:
{"name": "Alice Wonderland", "email": "alice@example.net"}

Extracted Data 2:
{"name": "Bob Builder", "email": "bob.builder@construction.com"}
```

Common Pitfalls:

- **Inconsistent examples:** Ensure your examples are perfectly consistent in format and content. Any deviation can confuse the model.
- **Too few examples:** For complex tasks, two or three examples might not be enough. Experiment with more if the model struggles.
- **Examples too long:** Be mindful of the context window. Long examples can quickly

consume your token budget.

Structured Outputs (JSON Mode)

Essence: Many LLM APIs, including Ollama, offer a

JSON mode or similar features to explicitly instruct the model to output valid JSON. This is invaluable for agents that need to parse model responses programmatically.

Why it matters for agents: Ensures reliable parsing of LLM outputs, allowing agents to extract data, trigger tools, or make decisions based on structured information without complex regex or string manipulation.

Practical Implementation:

Ollama supports a `format: json` option in its API calls.

Python

```
import ollama
import json

def get_json_output(prompt: str, model: str = "mistral") -> dict:
    try:
        response = ollama.chat(
            model=model,
            messages=[
                {
                    "role": "user",
                    "content": prompt,
                }
            ],
            format="json", # Crucial for JSON output
            options={
                "temperature": 0.01, # Keep temperature low for structured outp
            }
        )
        # Ollama returns the content as a string, even in JSON mode, so we pars
        return json.loads(response["message"]["content"])
    except json.JSONDecodeError as e:
        print(f"Error decoding JSON from LLM response: {e}")
        print(f"Raw LLM response: {response["message"]["content"]}")
        return {}
    except Exception as e:
        print(f"Error getting JSON output: {e}")
        return {}

if __name__ == "__main__":
    json_prompt = """
```

```
    Extract the following information from the text: product name, price, and c
    Text: "The new 'Quantum Leap' VR headset is available for $499.99."
    """
    extracted_data = get_json_output(json_prompt)
    print(f"Extracted JSON Data:\n{json.dumps(extracted_data, indent=2)}\n")

    json_prompt_2 = """
    List three benefits of using FastAPI for building APIs.
    Return the benefits as a JSON array of strings under the key 'benefits'.
    """
    fastapi_benefits = get_json_output(json_prompt_2)
    print(f"FastAPI Benefits:\n{json.dumps(fastapi_benefits, indent=2)}")
```

Test it:

1. Ensure `ollama serve` is running and `mistral` is pulled.
2. Save the Python code as `json_agent.py`.
3. Run `python3.11 json_agent.py`.

Expected Output: (Should be valid JSON)

```JSON
Extracted JSON Data:
{
  "product_name": "Quantum Leap VR headset",
  "price": 499.99,
  "currency": "$"
}

FastAPI Benefits:
{
  "benefits": [
    "High performance, comparable to NodeJS and Go",
    "Automatic interactive API documentation (Swagger UI and ReDoc)",
    "Type hints for better code completion and validation"
  ]
}
```

Common Pitfalls:

- **Model not adhering to JSON format:** While `format="json"` strongly encourages JSON, the model might still occasionally output malformed JSON, especially with complex instructions or if the prompt itself is ambiguous. Always include robust error handling (`try-except json.JSONDecodeError`).

- **Schema drift:** If your prompt implies a certain JSON schema, but the model outputs a slightly different one, your parsing logic might break. Be explicit in your prompt about the desired keys and value types.

3. Conversation Memory & State

For an AI agent to perform complex, multi-turn tasks, it needs memory. This memory allows it to maintain context, make informed decisions based on past interactions, and learn over time. Without memory, each interaction with the LLM would be stateless, like talking to someone who forgets everything you said a moment ago.

Pain Point Solved: Enabling agents to maintain context across multiple interactions, facilitating coherent conversations, complex reasoning, and stateful decision-making.

Why Agents Need Conversation History

Imagine an agent tasked with planning a trip. If it forgets the user's destination, dates, or preferences after each question, it would be useless. Conversation history provides:

- **Context:** The LLM can refer to previous turns to understand the current query.
- **Decision-Making:** Agents can build on past decisions or correct mistakes.
- **Learning (Implicit):** While not true long-term learning, the agent can adapt its responses within a single conversation based on user feedback.

Implementation Patterns

In-Memory Chat History (Simple, suitable for development)

For simple agents or during development, storing the conversation history in a Python list is sufficient. This is ephemeral and resets with each agent restart.

Practical Implementation:

Python

```
import ollama

class SimpleChatAgent:
    def __init__(self, model: str = "mistral"):
        self.model = model
        self.messages = [] # Stores conversation history

    def chat(self, user_message: str) -> str:
        self.messages.append({"role": "user", "content": user_message})
        try:
```

```
            response = ollama.chat(
                model=self.model,
                messages=self.messages,
            )
            agent_response = response["message"]["content"]
            self.messages.append({"role": "assistant", "content": agent_respons
            return agent_response
        except Exception as e:
            print(f"Error during chat: {e}")
            return "An error occurred."

if __name__ == "__main__":
    # Ensure 'ollama serve' is running and 'mistral' model is pulled
    agent = SimpleChatAgent()

    print("Agent: Hello! How can I help you today?")
    user_input = ""
    while user_input.lower() != "exit":
        user_input = input("You: ")
        if user_input.lower() != "exit":
            response = agent.chat(user_input)
            print(f"Agent: {response}")
```

Test it:

1. Ensure `ollama serve` is running and `mistral` is pulled.
2. Save the Python code as `simple_memory_agent.py`.
3. Run `python3.11 simple_memory_agent.py` and interact with it.

Expected Output: (A conversational flow where the agent remembers previous turns)

Plain Text

```
Agent: Hello! How can I help you today?
You: My name is John.
Agent: Nice to meet you, John. How can I assist you further?
You: What did I just tell you my name was?
Agent: You told me your name is John.
```

Common Pitfalls:

- **Ephemeral state:** Restarting the script loses all conversation history. Not suitable for production.
- **Context window limits:** As the conversation grows, the `messages` list can become very long, eventually exceeding the LLM's context window. This will lead to truncation or errors.

Persistent Storage (PostgreSQL, SQLite) for Production

For production agents, you need to store conversation history persistently. This allows agents to resume conversations, provides audit trails, and enables more sophisticated memory management.

Practical Implementation (SQLite Example):

We'll use SQLite for simplicity, but the principles apply to PostgreSQL or other databases. First, install `SQLAlchemy`:

Bash

```
sudo pip3 install sqlalchemy
```

Then, in your Python code (e.g., `persistent_memory_agent.py`):

Python

```
import ollama
import json
from sqlalchemy import create_engine, Column, Integer, String, Text
from sqlalchemy.orm import sessionmaker, declarative_base

# Database setup
DATABASE_URL = "sqlite:///./agent_memory.db"
engine = create_engine(DATABASE_URL)
Base = declarative_base()

class Conversation(Base):
    __tablename__ = "conversations"
    id = Column(Integer, primary_key=True, index=True)
    session_id = Column(String, index=True)
    role = Column(String)
    content = Column(Text)

Base.metadata.create_all(engine)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

class PersistentChatAgent:
    def __init__(self, session_id: str, model: str = "mistral"):
        self.session_id = session_id
        self.model = model
        self.db = SessionLocal()

    def _load_history(self) -> list:
        history = self.db.query(Conversation).filter(Conversation.session_id ==
        return [{"role": msg.role, "content": msg.content} for msg in history]
```

```
    def _save_message(self, role: str, content: str):
        new_message = Conversation(session_id=self.session_id, role=role, conte
        self.db.add(new_message)
        self.db.commit()
        self.db.refresh(new_message)

    def chat(self, user_message: str) -> str:
        # Load history for the current turn
        messages = self._load_history()
        messages.append({"role": "user", "content": user_message})
        self._save_message("user", user_message)

        try:
            response = ollama.chat(
                model=self.model,
                messages=messages,
            )
            agent_response = response["message"]["content"]
            self._save_message("assistant", agent_response)
            return agent_response
        except Exception as e:
            print(f"Error during chat: {e}")
            return "An error occurred."

    def __del__(self):
        self.db.close()

if __name__ == "__main__":
    # Ensure 'ollama serve' is running and 'mistral' model is pulled
    # Use a unique session ID for each conversation or user
    current_session_id = "user_123"
    agent = PersistentChatAgent(session_id=current_session_id)

    print(f"Agent (Session: {current_session_id}): Hello! How can I help you to
    user_input = ""
    while user_input.lower() != "exit":
        user_input = input("You: ")
        if user_input.lower() != "exit":
            response = agent.chat(user_input)
            print(f"Agent: {response}")

    # To demonstrate persistence, you can restart the script with the same sess
    # and the conversation history will be loaded.
```

Test it:

1. Ensure `ollama serve` is running and `mistral` is pulled.

2. Install `SQLAlchemy` (`sudo pip3 install sqlalchemy`).
3. Save the Python code as `persistent_memory_agent.py`.
4. Run `python3.11 persistent_memory_agent.py` and interact with it.
5. Exit the script, then run it again with the *same* `current_session_id`. The agent should remember your previous conversation.

Expected Output: (Similar to in-memory, but the history persists across script restarts)

Plain Text

```
Agent (Session: user_123): Hello! How can I help you today?
You: My favorite color is blue.
Agent: Blue is a lovely color! Is there anything specific you'd like to know or
You: What is my favorite color?
Agent: You mentioned your favorite color is blue.
```

Common Pitfalls:

- **Database connection issues:** Ensure your database is running and accessible. Check connection strings.
- **Schema migrations:** As your agent evolves, you might need to update your database schema. Use tools like Alembic for managing migrations in production.
- **Performance:** For very high-volume agents, frequent database reads/writes can become a bottleneck. Consider caching strategies or optimizing queries.

Vector-Based Memory Retrieval (Combining RAG + Conversation)

For agents that need to recall specific facts or details from a vast history or knowledge base, simply passing the entire raw conversation might be inefficient or exceed context limits. Vector-based memory retrieval, often part of a RAG system, allows the agent to intelligently retrieve *relevant* past interactions or documents.

Essence: Instead of sending the full history, the agent queries a vector database with the current turn and retrieves semantically similar past messages or external documents. These retrieved snippets are then added to the prompt.

Why it matters for agents: This is crucial for long-running agents, agents with access to large knowledge bases, or agents that need to avoid context window overflow while still having access to relevant information.

Practical Implementation (Conceptual Overview):

Implementing a full vector-based memory system involves:

1. **Embedding Generation:** Each user and agent message (or document chunk) is converted into a numerical vector (embedding) using an embedding model (which can also run locally via Ollama).
2. **Vector Database:** These embeddings are stored in a vector database (e.g., ChromaDB, FAISS, Pinecone).
3. **Retrieval:** When a new user message arrives, its embedding is generated, and a similarity search is performed against the vector database to find the most relevant past messages or documents.
4. **Augmentation:** The retrieved snippets are then included in the prompt sent to the LLM.

Python

```
# Conceptual Python Code (requires an embedding model and a vector DB library)
# This is an advanced topic, but illustrates the flow.

import ollama
# import chromadb # Example vector DB library

class VectorMemoryAgent:
    def __init__(self, session_id: str, model: str = "mistral", embedding_model
        self.session_id = session_id
        self.model = model
        self.embedding_model = embedding_model
        # self.vector_db = chromadb.Client() # Initialize your vector DB
        # self.collection = self.vector_db.get_or_create_collection(name=f"conv
        self.messages = [] # Still keep a short-term memory for immediate conte

    def _generate_embedding(self, text: str) -> list[float]:
        # Use Ollama to generate embeddings
        response = ollama.embeddings(model=self.embedding_model, prompt=text)
        return response["embedding"]

    def _retrieve_relevant_context(self, query: str, top_k: int = 3) -> list[st
        query_embedding = self._generate_embedding(query)
        # In a real system, you'd query your vector DB here
        # results = self.collection.query(query_embeddings=[query_embedding], n
        # For this conceptual example, we'll just return a placeholder
        return ["Retrieved context: User previously mentioned interest in Pytho
                "Retrieved context: Agent suggested learning FastAPI."]

    def chat(self, user_message: str) -> str:
        # Add user message to short-term memory
        self.messages.append({"role": "user", "content": user_message})

        # Retrieve relevant long-term context
```

```
        retrieved_context = self._retrieve_relevant_context(user_message)

        # Construct the prompt with retrieved context and short-term history
        full_prompt_messages = [
            {"role": "system", "content": "You are a helpful assistant. Use the
        ]
        for context_item in retrieved_context:
            full_prompt_messages.append({"role": "system", "content": context_i

        full_prompt_messages.extend(self.messages) # Add short-term conversatio

        try:
            response = ollama.chat(
                model=self.model,
                messages=full_prompt_messages,
            )
            agent_response = response["message"]["content"]
            self.messages.append({"role": "assistant", "content": agent_respons
            # In a real system, you'd also embed and store the new user/agent m
            return agent_response
        except Exception as e:
            print(f"Error during chat with vector memory: {e}")
            return "An error occurred."

if __name__ == "__main__":
    # This example is conceptual and requires a running vector DB and embedding
    # To run, you would need to install a vector DB client (e.g., pip install c
    # and pull an embedding model (ollama pull nomic-embed-text)
    print("Conceptual Vector Memory Agent. Requires vector DB setup to run full
    # agent = VectorMemoryAgent(session_id="user_456")
    # response = agent.chat("Tell me more about Python frameworks.")
    # print(f"Agent: {response}")
```

Common Pitfalls:

- **Complexity:** Setting up and managing a vector database adds significant complexity to your system.
- **Embedding model choice:** The quality of your embeddings directly impacts retrieval accuracy. Choose a suitable embedding model.
- **Over-retrieval/Under-retrieval:** Retrieving too much irrelevant information or too little relevant information can degrade performance. Fine-tune your retrieval strategy.

When to Trim/Summarize History (Token Limits, Performance)

Regardless of your memory strategy, managing the conversation history is crucial to stay within the LLM's context window and maintain performance. Strategies include:

- **Fixed Window:** Only keep the last N messages. Simple but can lose important context.
- **Summarization:** Periodically summarize older parts of the conversation using the LLM itself. This condenses information, but can lose detail.
- **Retrieval-Augmented:** As discussed, retrieve only relevant snippets from a larger memory store.

Practical Implementation (Summarization Example):

Python

```
import ollama

class SummarizingChatAgent:
    def __init__(self, model: str = "mistral", max_history_length: int = 5):
        self.model = model
        self.messages = []
        self.max_history_length = max_history_length

    def _summarize_history(self):
        if len(self.messages) > self.max_history_length:
            # Take the oldest messages (excluding the system prompt if any) and
            history_to_summarize = self.messages[:-self.max_history_length]
            current_conversation_part = self.messages[-self.max_history_length:

            summary_prompt = [
                {"role": "system", "content": "You are a helpful assistant. Sum
                *history_to_summarize,
                {"role": "user", "content": "Please provide a concise summary o
            ]
            try:
                summary_response = ollama.chat(model=self.model, messages=summa
                summary_content = summary_response["message"]["content"]
                # Replace old history with the summary and the recent messages
                self.messages = [
                    {"role": "system", "content": summary_content}
                ] + current_conversation_part
                print("--- History Summarized ---")
            except Exception as e:
                print(f"Error summarizing history: {e}", e)
                # Fallback: just trim if summarization fails
                self.messages = self.messages[-self.max_history_length:]

    def chat(self, user_message: str) -> str:
        self.messages.append({"role": "user", "content": user_message})

        # Summarize history before sending to LLM if it gets too long
        self._summarize_history()
```

```
        try:
            response = ollama.chat(
                model=self.model,
                messages=self.messages,
            )
            agent_response = response["message"]["content"]
            self.messages.append({"role": "assistant", "content": agent_respons
            return agent_response
        except Exception as e:
            print(f"Error during chat: {e}")
            return "An error occurred."

if __name__ == "__main__":
    # Ensure 'ollama serve' is running and 'mistral' model is pulled
    agent = SummarizingChatAgent(max_history_length=4) # Keep last 4 messages +

    print("Agent: Hello! Let's have a long chat.")
    user_inputs = [
        "My name is Alice and I work as a software engineer.",
        "I'm interested in learning about distributed systems.",
        "Can you recommend some good books on Kafka?",
        "What are the main challenges in implementing a microservices architect
        "Earlier, what did I say my profession was?"
    ]

    for user_input in user_inputs:
        print(f"You: {user_input}")
        response = agent.chat(user_input)
        print(f"Agent: {response}")
        print("\n---\n")

    # Observe the '--- History Summarized ---' message in the console
    # and how the agent still remembers Alice's profession due to summarization
```

Test it:

1. Ensure `ollama serve` is running and `mistral` is pulled.
2. Save the Python code as `summarizing_agent.py`.
3. Run `python3.11 summarizing_agent.py`.

Expected Output: You should see `--- History Summarized ---` appear in the console after a few turns, and the agent should still correctly recall Alice's profession, demonstrating that the summary preserved key information.

Common Pitfalls:

- **Loss of detail:** Summarization is lossy. Important nuances might be lost if the summary prompt isn't carefully crafted or if the LLM isn't good at summarization.
- **Cost/Latency of summarization:** Summarizing itself consumes tokens and adds latency. Balance the frequency of summarization with the benefits.
- **Prompt injection in summary:** Ensure the summary prompt is robust against malicious inputs that could alter the summary's intent.

4. MCP Server Integration (Advanced)

For an AI agent to truly interact with the world beyond generating text, it needs **tools**. The Model Context Protocol (MCP) provides a standardized way for agents to discover, invoke, and manage these tools, which can range from file system operations to external API calls. Think of MCP as the **operating system** for your agent, allowing it to run various applications (tools).

Pain Point Solved: Enabling agents to extend their capabilities beyond inherent LLM functions by providing a structured, interoperable mechanism to interact with external systems and execute code.

What MCPs Are and Why They're Critical for Agent Capability Extension

An MCP server exposes a set of capabilities (tools) that an AI agent can call. These capabilities are defined by schemas, allowing the agent (or rather, the LLM within the agent) to understand what a tool does, what inputs it requires, and what outputs to expect. This is the foundation of **function calling** in a robust agentic system.

Why it matters for agents: MCPs allow your Ollama-powered agents to:

- **Perform real-world actions:** Read/write files, execute code, interact with databases, call external APIs.
- **Be modular and extensible:** New tools can be added to an MCP server without modifying the core agent logic.
- **Operate reliably:** Tool schemas ensure the agent calls tools correctly, reducing errors.

Setting Up MCP Servers that Ollama Agents Can Call

An MCP server is essentially an API endpoint that exposes various functionalities. For a Python backend developer, this often means building a FastAPI application that defines and serves these tools.

Let's consider a simple MCP server that exposes file system operations.

Practical Implementation (FastAPI MCP Server):

First, install FastAPI and Uvicorn:

```bash
sudo pip3 install fastapi uvicorn
```

Create a file named `mcp_server.py`:

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import os

app = FastAPI(title="File System MCP Server", description="Exposes file system 

class ReadFileRequest(BaseModel):
    path: str

class WriteFileRequest(BaseModel):
    path: str
    content: str

class ListDirRequest(BaseModel):
    path: str

@app.post("/read_file")
async def read_file(request: ReadFileRequest):
    """Reads the content of a specified file."""
    try:
        with open(request.path, "r") as f:
            content = f.read()
        return {"status": "success", "content": content}
    except FileNotFoundError:
        raise HTTPException(status_code=404, detail=f"File not found: {request.
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error reading file: {e}")

@app.post("/write_file")
async def write_file(request: WriteFileRequest):
    """Writes content to a specified file. Creates the file if it doesn't exist
    try:
        with open(request.path, "w") as f:
            f.write(request.content)
        return {"status": "success", "message": f"File written to {request.path
    except Exception as e:
```

```
        raise HTTPException(status_code=500, detail=f"Error writing file: {e}")

@app.post("/list_directory")
async def list_directory(request: ListDirRequest):
    """Lists the contents of a specified directory."""
    try:
        contents = os.listdir(request.path)
        return {"status": "success", "contents": contents}
    except FileNotFoundError:
        raise HTTPException(status_code=404, detail=f"Directory not found: {req
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error listing directory:

# To run this server:
# uvicorn mcp_server:app --host 0.0.0.0 --port 8000
```

Test it:

1. Save the code above as `mcp_server.py`.
2. Run the server in a separate terminal: `uvicorn mcp_server:app --host 0.0.0.0 --port 8000`
3. Open your browser to `http://localhost:8000/docs` to see the auto-generated OpenAPI documentation (tool schemas).
4. You can test the endpoints using `curl` or a tool like Postman/Insomnia:

   Bash

   ```
   # Example: Write a file
   curl -X POST http://localhost:8000/write_file -H "Content-Type: application/

   # Example: Read the file
   curl -X POST http://localhost:8000/read_file -H "Content-Type: application/j

   # Example: List directory
   curl -X POST http://localhost:8000/list_directory -H "Content-Type: applicat
   ```

Common Pitfalls:

- **Server not running:** Your agent won't be able to call tools if the MCP server isn't active.
- **Incorrect JSON payload:** Ensure your requests match the Pydantic models defined in the FastAPI application.
- **Permissions issues:** The MCP server (and thus the agent) will only have the file system permissions of the user running the `uvicorn` process. Be mindful of security.

Connecting Ollama → MCP Server Communication Layer

Now, let's integrate this MCP server with our Ollama agent. The agent needs to be able to:

1. Understand what tools are available (by being provided with their schemas).
2. Decide when to use a tool.
3. Call the tool with the correct parameters.
4. Incorporate the tool's output back into its reasoning.

Ollama's `chat` endpoint supports a `tools` parameter where you can provide tool definitions. The LLM will then decide if and how to use them.

Practical Implementation (Ollama Agent with Tool Use):

Create a file named `ollama_tool_agent.py`:

Python

```
import ollama
import json
import requests

# Define the tools available to the agent
# These schemas are derived from your FastAPI MCP server's OpenAPI documentatio
tools = [
    {
        "type": "function",
        "function": {
            "name": "read_file",
            "description": "Reads the content of a specified file.",
            "parameters": {
                "type": "object",
                "properties": {
                    "path": {
                        "type": "string",
                        "description": "The absolute path to the file to read."
                    }
                },
                "required": ["path"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "write_file",
            "description": "Writes content to a specified file. Creates the fil
            "parameters": {
                "type": "object",
```

```
                "properties": {
                    "path": {
                        "type": "string",
                        "description": "The absolute path to the file to write.
                    },
                    "content": {
                        "type": "string",
                        "description": "The content to write to the file."
                    }
                },
                "required": ["path", "content"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "list_directory",
            "description": "Lists the contents of a specified directory.",
            "parameters": {
                "type": "object",
                "properties": {
                    "path": {
                        "type": "string",
                        "description": "The absolute path to the directory to l
                    }
                },
                "required": ["path"]
            }
        }
    }
]

class ToolUsingAgent:
    def __init__(self, model: str = "mistral", mcp_server_url: str = "http://lo
        self.model = model
        self.mcp_server_url = mcp_server_url
        self.messages = []

    def call_tool(self, tool_name: str, **kwargs):
        """Calls the appropriate tool on the MCP server."""
        endpoint = f"{self.mcp_server_url}/{tool_name}"
        try:
            response = requests.post(endpoint, json=kwargs)
            response.raise_for_status() # Raise an exception for HTTP errors
            return response.json()
        except requests.exceptions.RequestException as e:
            return {"status": "error", "message": f"Tool call failed: {e}"}
```

```
    def chat(self, user_message: str) -> str:
        self.messages.append({"role": "user", "content": user_message})

        try:
            response = ollama.chat(
                model=self.model,
                messages=self.messages,
                tools=tools, # Provide the tool definitions to Ollama
            )

            # Check if the LLM decided to call a tool
            if response["message"].get("tool_calls"):
                tool_calls = response["message"]["tool_calls"]
                for tool_call in tool_calls:
                    function_name = tool_call["function"]["name"]
                    function_args = tool_call["function"]["arguments"]

                    print(f"Agent decided to call tool: {function_name} with ar

                    # Call the actual tool via the MCP server
                    tool_output = self.call_tool(function_name, **function_args
                    print(f"Tool output: {tool_output}")

                    # Add tool output back to messages for the LLM to process
                    self.messages.append({
                        "role": "tool",
                        "content": json.dumps(tool_output),
                        "tool_call_id": tool_call["id"]
                    })

                # After calling tools, make another call to the LLM to get a na
                # based on the tool output.
                final_response = ollama.chat(
                    model=self.model,
                    messages=self.messages,
                )
                agent_response = final_response["message"]["content"]
                selflama.messages.append({"role": "assistant", "content": agent_
                return agent_response
            else:
                # If no tool call, it's a regular chat response
                agent_response = response["message"]["content"]
                self.messages.append({"role": "assistant", "content": agent_res
                return agent_response

        except Exception as e:
            print(f"Error during chat with tool use: {e}")
```

```
            return "An error occurred."

if __name__ == "__main__":
    # Ensure 'ollama serve' is running and 'mistral' model is pulled
    # Ensure 'mcp_server.py' is running with 'uvicorn mcp_server:app --host 0.0

    agent = ToolUsingAgent()

    print("Agent: Hello! I can help you with file operations.")
    user_input = ""
    while user_input.lower() != "exit":
        user_input = input("You: ")
        if user_input.lower() != "exit":
            response = agent.chat(user_input)
            print(f"Agent: {response}")
```

Test it:

1. Ensure `ollama serve` is running (`ollama pull mistral`).
2. Ensure your `mcp_server.py` is running in a separate terminal (`uvicorn mcp_server:app --host 0.0.0.0 --port 8000`).
3. Save the Python code above as `ollama_tool_agent.py` .
4. Run `python3.11 ollama_tool_agent.py` .

Example Interaction:

Plain Text

```
Agent: Hello! I can help you with file operations.
You: Create a file named 'my_document.txt' in the /tmp directory with the conte
Agent decided to call tool: write_file with args: {'path': '/tmp/my_document.tx
Tool output: {'status': 'success', 'message': 'File written to /tmp/my_document
Agent: I have successfully created the file 'my_document.txt' in the /tmp direc
You: What are the contents of /tmp/my_document.txt?
Agent decided to call tool: read_file with args: {'path': '/tmp/my_document.txt
Tool output: {'status': 'success', 'content': 'This is a test document.'}
Agent: The file '/tmp/my_document.txt' contains: "This is a test document."
You: List the files in the /tmp directory.
Agent decided to call tool: list_directory with args: {'path': '/tmp'}
Tool output: {'status': 'success', 'contents': ['test_agent_file.txt', 'my_docu
Agent: The files in the /tmp directory are: test_agent_file.txt, my_document.tx
```

Common Pitfalls:

- **Tool schema accuracy:** The `tools` list provided to Ollama *must* accurately reflect the actual API of your MCP server. Any mismatch will lead to the LLM generating incorrect

tool calls.

- **LLM not calling tools:** If the LLM doesn't seem to call tools, ensure your prompt clearly indicates that tools are available and that the task requires tool use. Sometimes, adding a system prompt like "You have access to the following tools: [list tools]. Use them when necessary." can help.
- **Error handling:** Robust error handling in `call_tool` and around the `ollama.chat` calls is crucial for stable agents.

Building a Custom MCP Server for Your Use Case

Extending your agent's capabilities is as simple as adding new endpoints to your FastAPI MCP server and updating the `tools` definition in your agent code. You can create tools for:

- **Code Execution:** An endpoint that takes code as input, executes it (e.g., in a sandboxed environment), and returns the output.
- **External API Integrations:** Wrap calls to third-party APIs (e.g., weather API, stock market data, email service) into tool functions.
- **Database Operations:** Tools to query or modify data in a database.

Pain Point Solved: Providing a flexible and scalable way to give agents access to virtually any external capability, transforming them from mere conversational interfaces into powerful automation engines.

Error Handling and Tool Schema Validation

Robust agents require careful error handling. When an LLM calls a tool, several things can go wrong:

- **Malformed tool call:** The LLM might generate arguments that don't match the schema.
- **Tool execution failure:** The external system might return an error (e.g., file not found, API rate limit).
- **Network issues:** The MCP server might be unreachable.

Your agent's `call_tool` function should gracefully handle these. The `tool` role in the `messages` list is where you feed back the result (or error) of the tool execution to the LLM, allowing it to adapt or inform the user.

Why it matters for agents: Ensures the agent can recover from failures, provide meaningful feedback to the user, and maintain operational stability in complex environments.

5. Building End-to-End Agent Applications

Now that we've covered the foundational components, let's see how they come together to build complete, functional AI agents. An agentic application is more than just an LLM; it's an orchestration of models, memory, and tools working in concert to achieve a goal.

Architecture Pattern: Ollama LLM + MCP Servers + State Management + Tools

The typical architecture for a robust Ollama-powered agentic system looks like this:

1. **Ollama LLM (The Brain):** Runs locally, providing the core reasoning, language understanding, and generation capabilities. It interprets user requests, plans actions, and formulates responses.
2. **State Management (The Memory):** Handles conversation history and long-term memory. This can be in-memory for simple cases, persistent storage (like a database) for production, or vector-based for advanced RAG and knowledge retrieval.
3. **MCP Servers (The Hands & Feet):** External services (often FastAPI applications) that expose specific tools. These tools allow the agent to interact with the environment (file system, web, external APIs, code execution).
4. **Tools (The Actions):** The specific functions or capabilities exposed by the MCP servers, defined by schemas that the LLM can understand and invoke.
5. **Orchestration Logic (The Conductor):** Your Python code that ties everything together. It receives user input, passes it to the LLM, interprets the LLM's intent (e.g., tool call or direct response), executes tools if necessary, and manages the conversation flow.

This modular design allows for flexibility, scalability, and clear separation of concerns, making your agents easier to develop, debug, and maintain.

Project Example 1: Web Scraper & Summarizer Agent

Let's build an agent that can scrape content from a URL, summarize it, and answer questions about the content. This agent will demonstrate tool use (web scraping), LLM summarization, and potentially RAG for answering questions.

Goal: Given a URL, the agent should be able to fetch its content, summarize it, and then answer follow-up questions about the content.

Components:

- **Ollama LLM:** For summarization and question answering.
- **MCP Server (Web Scraper Tool):** A FastAPI endpoint that takes a URL and returns its text content.
- **State Management:** In-memory for simplicity, but could be extended to persistent or

vector memory.

Step 1: Create the Web Scraper MCP Server

We'll extend our `mcp_server.py` or create a new one. For this example, let's create a new file `web_scraper_mcp.py`.

Bash

```
sudo pip3 install beautifulsoup4 requests
```

`web_scraper_mcp.py`:

Python

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import requests
from bs4 import BeautifulSoup

app = FastAPI(title="Web Scraper MCP Server", description="Exposes web scraping

class ScrapeURLRequest(BaseModel):
    url: str

@app.post("/scrape_url")
async def scrape_url(request: ScrapeURLRequest):
    """Fetches the text content from a given URL."""
    try:
        headers = {
            "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKi
        }
        response = requests.get(request.url, headers=headers, timeout=10)
        response.raise_for_status() # Raise an exception for HTTP errors
        soup = BeautifulSoup(response.text, "html.parser")
        # Extract text, removing script and style elements
        for script_or_style in soup(["script", "style"]):
            script_or_style.extract()
        text_content = soup.get_text(separator=" ", strip=True)
        return {"status": "success", "content": text_content}
    except requests.exceptions.RequestException as e:
        raise HTTPException(status_code=500, detail=f"Error fetching URL: {e}")
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error processing content:
```

```
# To run this server:
# uvicorn web_scraper_mcp:app --host 0.0.0.0 --port 8001
```

Test it:

1. Save the code as `web_scraper_mcp.py`.
2. Run the server in a separate terminal: `uvicorn web_scraper_mcp:app --host 0.0.0.0 --port 8001`
3. Test with `curl`:

```bash
curl -X POST http://localhost:8001/scrape_url -H "Content-Type: application/
```

Step 2: Create the Web Agent

Now, let's create an agent that uses this web scraping tool.

`web_agent.py`:

```python
import ollama
import json
import requests

# Define the tools available to the agent
tools = [
    {
        "type": "function",
        "function": {
            "name": "scrape_url",
            "description": "Fetches the text content from a given URL.",
            "parameters": {
                "type": "object",
                "properties": {
                    "url": {
                        "type": "string",
                        "description": "The URL to scrape."
                    }
                },
                "required": ["url"]
            }
        }
    }
]
```

```
class WebScraperAgent:
    def __init__(self, model: str = "mistral", scraper_mcp_url: str = "http://l
        self.model = model
        self.scraper_mcp_url = scraper_mcp_url
        self.messages = []
        self.scraped_content = "" # To store the last scraped content

    def call_tool(self, tool_name: str, **kwargs):
        """Calls the appropriate tool on the MCP server."""
        endpoint = f"{self.scraper_mcp_url}/{tool_name}"
        try:
            response = requests.post(endpoint, json=kwargs)
            response.raise_for_status() # Raise an exception for HTTP errors
            return response.json()
        except requests.exceptions.RequestException as e:
            return {"status": "error", "message": f"Tool call failed: {e}"}

    def chat(self, user_message: str) -> str:
        self.messages.append({"role": "user", "content": user_message})

        try:
            response = ollama.chat(
                model=self.model,
                messages=self.messages,
                tools=tools,
            )

            if response["message"].get("tool_calls"):
                tool_calls = response["message"]["tool_calls"]
                for tool_call in tool_calls:
                    function_name = tool_call["function"]["name"]
                    function_args = tool_call["function"]["arguments"]

                    print(f"Agent decided to call tool: {function_name} with ar

                    tool_output = self.call_tool(function_name, **function_args
                    print(f"Tool output: {tool_output}")

                    if function_name == "scrape_url" and tool_output.get("statu
                        self.scraped_content = tool_output["content"]
                        # Limit content sent back to LLM to avoid context windo
                        content_preview = self.scraped_content[:1000] + "..." i
                        tool_output["content_preview"] = content_preview
                        del tool_output["content"] # Don't send full content ba

                    self.messages.append({
                        "role": "tool",
                        "content": json.dumps(tool_output),
```

```
                    "tool_call_id": tool_call["id"]
                })

            # After calling tools, make another call to the LLM to get a na
            final_response = ollama.chat(
                model=self.model,
                messages=self.messages,
            )
            agent_response = final_response["message"]["content"]
            self.messages.append({"role": "assistant", "content": agent_res
            return agent_response
        else:
            # If no tool call, it's a regular chat response
            agent_response = response["message"]["content"]
            self.messages.append({"role": "assistant", "content": agent_res
            return agent_response

    except Exception as e:
        print(f"Error during chat with tool use: {e}")
        return "An error occurred."

def summarize_scraped_content(self) -> str:
    if not self.scraped_content:
        return "No content has been scraped yet to summarize."

    summary_prompt = [
        {"role": "system", "content": "You are a helpful assistant. Summari
        {"role": "user", "content": f"Summarize the following text:\n\n{sel
    ]
    try:
        response = ollama.chat(
            model=self.model,
            messages=summary_prompt,
            options={
                "num_predict": 500 # Limit summary length
            }
        )
        return response["message"]["content"]
    except Exception as e:
        print(f"Error summarizing content: {e}")
        return "An error occurred during summarization."

def answer_question_about_scraped_content(self, question: str) -> str:
    if not self.scraped_content:
        return "No content has been scraped yet to answer questions about."

    qa_prompt = [
        {"role": "system", "content": "You are a helpful assistant. Answer
```

```
            {"role": "user", "content": f"Text:\n{self.scraped_content}\n\nQues
        ]
        try:
            response = ollama.chat(
                model=self.model,
                messages=qa_prompt,
                options={
                    "num_predict": 200 # Limit answer length
                }
            )
            return response["message"]["content"]
        except Exception as e:
            print(f"Error answering question: {e}")
            return "An error occurred while answering the question."

if __name__ == "__main__":
    # Ensure 'ollama serve' is running and 'mistral' model is pulled
    # Ensure 'web_scraper_mcp.py' is running with 'uvicorn web_scraper_mcp:app

    agent = WebScraperAgent()

    print("Agent: Hello! I can scrape websites, summarize them, and answer ques
    user_input = ""
    while user_input.lower() != "exit":
        user_input = input("You: ")
        if user_input.lower() == "exit":
            break

        if "scrape" in user_input.lower() and "http" in user_input.lower( ):
            response = agent.chat(user_input)
            print(f"Agent: {response}")
        elif "summarize" in user_input.lower():
            summary = agent.summarize_scraped_content()
            print(f"Agent Summary: {summary}")
        elif "question about" in user_input.lower():
            question = user_input.split("question about", 1)[1].strip()
            answer = agent.answer_question_about_scraped_content(question)
            print(f"Agent Answer: {answer}")
        else:
            response = agent.chat(user_input)
            print(f"Agent: {response}")
```

Test it:

1. Ensure `ollama serve` is running (`ollama pull mistral`).
2. Ensure your `web_scraper_mcp.py` is running in a separate terminal (`uvicorn web_scraper_mcp:app --host 0.0.0.0 --port 8001`).

3. Save the Python code above as `web_agent.py`.
4. Run `python3.11 web_agent.py`.

Example Interaction:

```
Agent: Hello! I can scrape websites, summarize them, and answer questions.
You: Please scrape the content from https://www.google.com
Agent decided to call tool: scrape_url with args: {"url": "https://www.google.c
Tool output: {"status": "success", "content_preview": "..."}
Agent: I have successfully scraped the content from https://www.google.com. Wha
You: Summarize the content.
Agent Summary: The content from Google.com is primarily the Google search engin
You: What is the main purpose of the page?
Agent Answer: The main purpose of the page is to provide a search engine interf
```

Common Pitfalls:

- **Website blocking scrapers:** Some websites actively block automated scraping. Your agent might receive HTTP 403 (Forbidden) errors. Using more sophisticated headers or rotating proxies can help, but is beyond this basic example.
- **Content parsing:** Extracting clean text from HTML can be tricky. `BeautifulSoup` helps, but complex layouts might require more specific parsing logic.
- **Context window for scraped content:** Large articles will exceed the LLM's context window. You'll need to implement strategies like chunking and RAG (as discussed in Vector-Based Memory) to handle this effectively.

Performance Optimization (Batching, Caching, Async Operations)

For production-grade agents, performance is key. Here are some strategies:

- **Batching:** If your agent needs to process multiple independent requests, batching them can reduce overhead. Instead of making N individual LLM calls, combine them into one larger call if the model supports it.
- **Caching:** Cache LLM responses for identical prompts. If the agent asks the same question or performs the same summarization multiple times, return the cached result instead of re-querying the LLM.
- **Asynchronous Operations:** Use Python's `asyncio` with `FastAPI` and `httpx` (for async HTTP requests) to handle multiple agent requests concurrently, preventing your agent from blocking while waiting for LLM responses or tool calls.

Pain Point Solved: Ensuring agents are responsive, efficient, and can handle high loads

without degrading user experience or incurring excessive costs.

Practical Implementation (Async Example - Conceptual):

```python
import ollama
import asyncio
import httpx # For async HTTP requests

# Assuming an async version of ollama client or using httpx directly
# ollama-python library might not be fully async for chat calls yet, but the pr

async def async_get_llm_response(prompt: str, model: str = "mistral" ) -> str:
    # In a real async setup, you'd use an async Ollama client or httpx to call
    # For demonstration, we'll simulate async behavior
    await asyncio.sleep(0.1 ) # Simulate network/processing delay
    # response = await ollama.AsyncClient().chat(model=model, messages=[...])
    return f"Async response for: {prompt[:30]}..."

async def main():
    prompts = [
        "What is the capital of France?",
        "Who wrote 'Pride and Prejudice'?",
        "Explain quantum entanglement in simple terms."
    ]

    tasks = [async_get_llm_response(p) for p in prompts]
    results = await asyncio.gather(*tasks)

    for i, result in enumerate(results):
        print(f"Prompt {i+1}: {result}")

if __name__ == "__main__":
    asyncio.run(main())
```

Common Pitfalls:

- **Over-optimization:** Don't optimize prematurely. Start with a working agent and identify bottlenecks before implementing complex caching or batching.
- **Complexity of async:** Asynchronous programming adds complexity. Ensure your team is comfortable with `asyncio`.

Testing and Validation Strategies for Agents

Testing agents is more complex than traditional software due to the probabilistic nature of LLMs. You need to validate both the deterministic parts (tool calls, data parsing) and the

non-deterministic parts (LLM reasoning, response quality).

- **Unit Tests:** For individual tools, data parsing logic, and state management components.
- **Integration Tests:** Verify that the agent correctly calls tools, processes their output, and generates a coherent response.
- **End-to-End Tests:** Simulate user interactions and evaluate the agent's overall performance against predefined scenarios and expected outcomes.
- **Golden Set Testing:** Create a

  set of expected inputs and their corresponding ideal outputs. Run your agent against this set regularly to detect regressions.
- **Human-in-the-Loop (HITL):** For critical applications, human oversight and correction can be integrated into the agent's workflow, especially for edge cases or ambiguous situations.

Pain Point Solved: Ensuring the reliability, accuracy, and robustness of agentic applications, which is challenging due to the non-deterministic nature of LLMs.

Project Example 2: Document Processor Agent (Extracting Structured Data)

Let's build an agent that can process a document (e.g., a text file), extract specific structured information, and then present it. This agent will demonstrate file system tool use and structured output capabilities.

Goal: Given a text file containing semi-structured data (e.g., product information), the agent should read the file, extract key-value pairs (product name, price, description), and return them as JSON.

Components:

- **Ollama LLM:** For structured data extraction.
- **MCP Server (File System Tool):** Our existing `mcp_server.py` with `read_file` capability.
- **State Management:** In-memory for simplicity.

Step 1: Ensure File System MCP Server is Running

Make sure your `mcp_server.py` (from Section 4) is running on `http://localhost:8000`.

```bash
uvicorn mcp_server:app --host 0.0.0.0 --port 8000
```

Step 2: Create a Sample Document

Let's create a sample product information file. Save this as `/tmp/product_info.txt`:

Plain Text

```
Product Name: SuperWidget Pro
Price: $199.99
Description: A high-performance widget designed for professional use. Features
Availability: In Stock
SKU: SWP-2024-001
```

Step 3: Create the Document Processor Agent

`document_agent.py`:

Python

```
import ollama
import json
import requests

# Define the tools available to the agent (only read_file needed from mcp_serve
tools = [
    {
        "type": "function",
        "function": {
            "name": "read_file",
            "description": "Reads the content of a specified file.",
            "parameters": {
                "type": "object",
                "properties": {
                    "path": {
                        "type": "string",
                        "description": "The absolute path to the file to read."
                    }
                },
                "required": ["path"]
            }
        }
    }
]

class DocumentProcessorAgent:
    def __init__(self, model: str = "mistral", mcp_server_url: str = "http://lo
        self.model = model
        self.mcp_server_url = mcp_server_url
```

```
        self.messages = []
        self.document_content = "" # To store the content of the last read docu

    def call_tool(self, tool_name: str, **kwargs):
        """Calls the appropriate tool on the MCP server."""
        endpoint = f"{self.mcp_server_url}/{tool_name}"
        try:
            response = requests.post(endpoint, json=kwargs)
            response.raise_for_status() # Raise an exception for HTTP errors
            return response.json()
        except requests.exceptions.RequestException as e:
            return {"status": "error", "message": f"Tool call failed: {e}"}

    def chat(self, user_message: str) -> str:
        self.messages.append({"role": "user", "content": user_message})

        try:
            response = ollama.chat(
                model=self.model,
                messages=self.messages,
                tools=tools,
            )

            if response["message"].get("tool_calls"):
                tool_calls = response["message"]["tool_calls"]
                for tool_call in tool_calls:
                    function_name = tool_call["function"]["name"]
                    function_args = tool_call["function"]["arguments"]

                    print(f"Agent decided to call tool: {function_name} with ar

                    tool_output = self.call_tool(function_name, **function_args
                    print(f"Tool output: {tool_output}")

                    if function_name == "read_file" and tool_output.get("status
                        self.document_content = tool_output["content"]

                    self.messages.append({
                        "role": "tool",
                        "content": json.dumps(tool_output),
                        "tool_call_id": tool_call["id"]
                    })

                # After calling tools, make another call to the LLM to get a na
                final_response = ollama.chat(
                    model=self.model,
                    messages=self.messages,
                )
```

```
                agent_response = final_response["message"]["content"]
                self.messages.append({"role": "assistant", "content": agent_res
                return agent_response
            else:
                # If no tool call, it's a regular chat response
                agent_response = response["message"]["content"]
                self.messages.append({"role": "assistant", "content": agent_res
                return agent_response

        except Exception as e:
            print(f"Error during chat with tool use: {e}")
            return "An error occurred."

    def extract_product_info(self) -> dict:
        if not self.document_content:
            return {"error": "No document content loaded to extract from."}

        extraction_prompt = [
            {"role": "system", "content": "You are an expert data extractor. Ex
            {"role": "user", "content": f"Extract information from the followin
        ]
        try:
            response = ollama.chat(
                model=self.model,
                messages=extraction_prompt,
                format="json",
                options={
                    "temperature": 0.01 # Keep low for accurate extraction
                }
            )
            return json.loads(response["message"]["content"])
        except json.JSONDecodeError as e:
            print(f"Error decoding JSON from LLM response during extraction: {e
            return {"error": "Failed to parse JSON from LLM.", "raw_response":
        except Exception as e:
            print(f"Error during product info extraction: {e}")
            return {"error": "An error occurred during extraction."}

if __name__ == "__main__":
    # Ensure 'ollama serve' is running and 'mistral' model is pulled
    # Ensure 'mcp_server.py' is running with 'uvicorn mcp_server:app --host 0.0

    agent = DocumentProcessorAgent()

    print("Agent: Hello! I can process documents and extract structured informa
    user_input = ""
    while user_input.lower() != "exit":
        user_input = input("You: ")
```

```
        if user_input.lower() == "exit":
            break

        if "read file" in user_input.lower() and "/tmp/product_info.txt" in use
            response = agent.chat(user_input) # This will trigger the read_file
            print(f"Agent: {response}")
            if agent.document_content:
                print("Agent: Document loaded. Now extracting product informati
                extracted_data = agent.extract_product_info()
                print(f"Extracted Product Info:\n{json.dumps(extracted_data, in
        else:
            response = agent.chat(user_input)
            print(f"Agent: {response}")
```

Test it:

1. Ensure `ollama serve` is running (`ollama pull mistral`).
2. Ensure your `mcp_server.py` is running in a separate terminal (`uvicorn mcp_server:app --host 0.0.0.0 --port 8000`).
3. Create the `/tmp/product_info.txt` file as described above.
4. Save the Python code above as `document_agent.py` .
5. Run `python3.11 document_agent.py` .

Example Interaction:

Plain Text

```
Agent: Hello! I can process documents and extract structured information.
You: Please read the file /tmp/product_info.txt
Agent decided to call tool: read_file with args: {"path": "/tmp/product_info.tx
Tool output: {"status": "success", "content": "Product Name: SuperWidget Pro\nP
Agent: I have read the file /tmp/product_info.txt. It contains product informat
Agent: Document loaded. Now extracting product information...
Extracted Product Info:
{
  "product_name": "SuperWidget Pro",
  "price": "$199.99",
  "description": "A high-performance widget designed for professional use. Feat
  "availability": "In Stock",
  "sku": "SWP-2024-001"
}
```

Common Pitfalls:

- **LLM extraction accuracy:** For highly unstructured or complex documents, the LLM might struggle to extract information perfectly. Fine-tuning the prompt, providing more

examples, or using more advanced models can help.

- **Error handling for JSON:** Always anticipate that the LLM might return malformed JSON and include `try-except json.JSONDecodeError`.

6. Work Beyond Expectations

To truly master agentic AI with Ollama, consider these advanced techniques and considerations:

Prompt Engineering Techniques for Maximizing Model Capability

- **Chain-of-Thought (CoT) Prompting:** Encourage the LLM to

  show its reasoning process step-by-step before providing the final answer. This often leads to more accurate and reliable results, especially for complex tasks.
- **Self-Correction:** Design prompts that allow the agent to evaluate its own output or tool calls and correct mistakes. This can involve asking the LLM to critique its previous response or to re-evaluate a tool call if the initial output was unexpected.
- **Role-Playing:** Assign specific roles to the LLM within the prompt (e.g., "You are an expert Python developer...") to elicit more focused and high-quality responses.

Multi-turn Reasoning (Chain-of-Thought, Self-Correction)

Agents often need to perform tasks that require multiple steps of reasoning and interaction. This is where multi-turn reasoning comes into play.

Essence: Instead of a single prompt-response cycle, the agent engages in an internal monologue or a series of interactions with the LLM, breaking down complex problems into smaller, manageable steps.

Why it matters for agents: Enables agents to tackle highly complex problems that cannot be solved in a single LLM call, mimicking human problem-solving approaches.

Practical Implementation (Conceptual):

Python

```
# Conceptual example of multi-turn reasoning

def multi_step_reasoning_agent(initial_query: str, model: str = "mistral") -> s
    thought_process = []
    current_state = {"query": initial_query, "context": ""}

    # Step 1: Initial understanding and plan
    plan_prompt = f"""
```

```
    You are an AI assistant. The user has a query: "{current_state["query"]}".
    First, analyze the query and outline a step-by-step plan to address it.
    Output your plan as a numbered list.
    """
    plan_response = ollama.chat(model=model, messages=[{"role": "user", "conten
    plan = plan_response["message"]["content"]
    thought_process.append(f"Plan: {plan}")
    print(f"Agent Plan:\n{plan}\n")

    # Step 2: Execute first step (e.g., search for information)
    # This would involve calling a tool based on the plan
    # For demonstration, let's simulate a search result
    simulated_search_result = "According to Wikipedia, the capital of France is
    thought_process.append(f"Executed step 1: Retrieved information: {simulated_
    current_state["context"] += simulated_search_result

    # Step 3: Synthesize information and answer
    synthesis_prompt = f"""
    Based on the following context and your plan:
    Context: {current_state["context"]}
    Plan: {plan}
    Answer the user's original query: "{current_state["query"]}".
    """
    final_answer_response = ollama.chat(model=model, messages=[{"role": "user",
    final_answer = final_answer_response["message"]["content"]
    thought_process.append(f"Final Answer: {final_answer}")

    return final_answer

if __name__ == "__main__":
    # Ensure ollama serve is running and mistral model is pulled
    # print(multi_step_reasoning_agent("What is the capital of France and why i
    print("Multi-turn reasoning is conceptual here; requires more complex orche
```

Tool Use Optimization (Knowing which tools to call, when, and why)

As agents gain more tools, the challenge shifts from *how* to use a tool to *which* tool to use. Effective tool use optimization involves:

- **Clear Tool Descriptions:** Ensure your tool schemas and descriptions are unambiguous, so the LLM can accurately determine their applicability.
- **Tool Orchestration Frameworks:** Libraries like LangChain or LlamaIndex provide abstractions for managing tool selection and execution, often using an LLM to decide the tool-use chain.
- **Feedback Loops:** Allow the agent to learn from successful and unsuccessful tool calls,

refining its tool-use strategy over time.

Handling Edge Cases and Failures Gracefully

Production agents will encounter unexpected inputs, tool failures, and ambiguous situations. Robust error handling is paramount:

- **Input Validation:** Validate user inputs before passing them to the LLM or tools.
- **Tool Call Validation:** Before executing a tool call suggested by the LLM, validate its parameters against the tool schema.
- **Retry Mechanisms:** Implement retries for transient errors (e.g., network issues when calling an MCP server).
- **Fallback Strategies:** If a tool fails or the LLM provides an unparseable response, have a fallback (e.g., inform the user, try a different approach, use a simpler LLM call).
- **Human-in-the-Loop:** For critical failures or ambiguous situations, escalate to a human operator.

END GOAL: What You Should Be Able To Do

After this learning session, you should be able to:

- [ ] Install and run Ollama models confidently, choosing the right model for your task.
- [ ] Build a Python FastAPI agent that calls Ollama and maintains conversation context.
- [ ] Connect multiple MCP servers to an Ollama agent for real-world work (file ops, code execution, deployments).
- [ ] Implement persistent conversation memory (in-memory for dev, PostgreSQL for prod).
- [ ] Design and build a working agent application that demonstrates file/code operations, MCP integration, and conversation memory.
- [ ] Debug and optimize agent performance (latency, token usage, accuracy).
- [ ] Explain trade-offs between local Ollama models vs. API-based LLMs for your specific use case.

NARROWING: Important Considerations

- No LLM pretraining, no fine-tuning (that's a separate advanced topic).
- No vector database theory; focus on practical RAG with Ollama + embeddings.

- Skip low-level MCP protocol details; focus on *using* MCP servers, not building from scratch initially.
- Avoid abstract architecture diagrams; show working code patterns instead.
- Focus on models that run on consumer hardware (assume CPU or modest GPU).
- All examples should be runnable in <5 minutes on a fresh Ollama install.
- Prioritize Ollama's *current stable features*, not experimental/bleeding-edge.

VALIDATION:

- Every concept should include a "test it" section (command or code snippet).
- Every tool integration should show expected output and common errors.
- Every code example should be copy-paste ready (with comments).

References

- Ollama Documentation - Official documentation for installing, running, and interacting with Ollama.
- FastAPI Documentation - Documentation for building web APIs in Python, used for MCP servers.
- SQLAlchemy Documentation - Python SQL toolkit and Object Relational Mapper, used for persistent memory.
- BeautifulSoup Documentation - Python library for parsing HTML and XML documents, used for web scraping.
- Requests Library Documentation - HTTP library for Python, used for making API calls.
- Ollama Python Library - Official Python client for Ollama.